

# Introduction - Analyse de données et Probabilités

Arsène

## Introduction

Ce projet explore différents aspects de la statistique : de la théorie des lois de probabilité à l'analyse de données réelles (Keno) en s'inspirant de l'article @<https://doi.org/10.1111/test.12409>.

## 1. Probabilités Théoriques

### Loi de Poisson

La loi de Poisson est une loi de probabilité **discrète** qui décrit le nombre d'événements se produisant dans un intervalle de temps ou d'espace fixe.

Si  $X \sim \mathcal{P}(\lambda)$ , alors :

$$P(X = x) = \frac{\lambda^x}{x!} e^{-\lambda}, \quad x \in \mathbb{N}$$

```
x <- 0:10
dx <- dpois(x, lambda = 3)

plot(x, dx, type = "h", lwd = 2, col = "steelblue",
     main = "Loi de Poisson ( λ = 3)",
     xlab = "Nombre d'événements", ylab = "Probabilité")
points(x, dx, pch = 16, col = "steelblue")
```

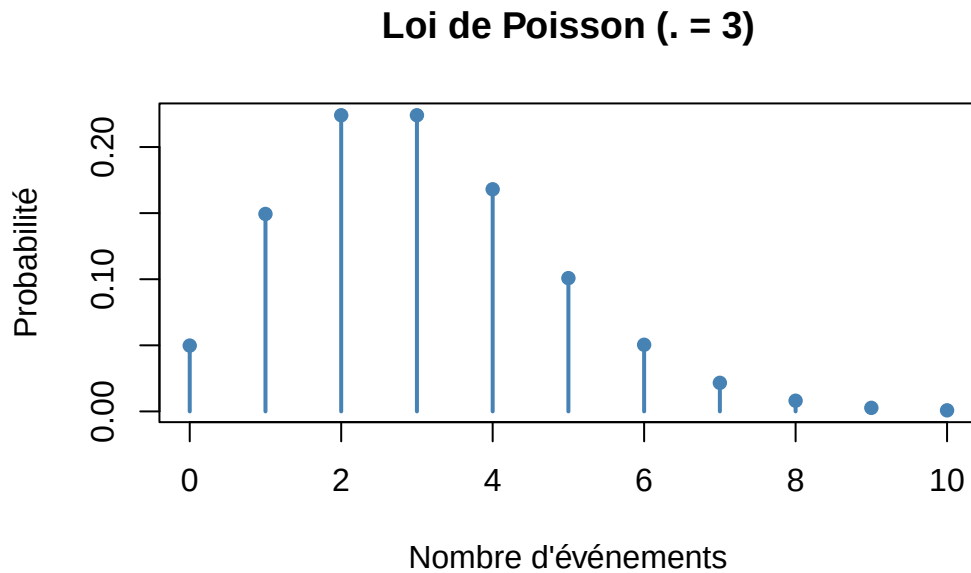


FIGURE 1 : Distribution de Poisson

### L'espérance mathématique

L'espérance mathématique, c'est la **valeur moyenne théorique** d'une variable aléatoire

```
# L'espérance c'est la somme des x * P(X=x)
esperance <- sum(x * dx)
print(paste("Espérance = ", round(esperance, 3)))
```

```
[1] "Espérance = 2.997"
```

$$\approx 3$$

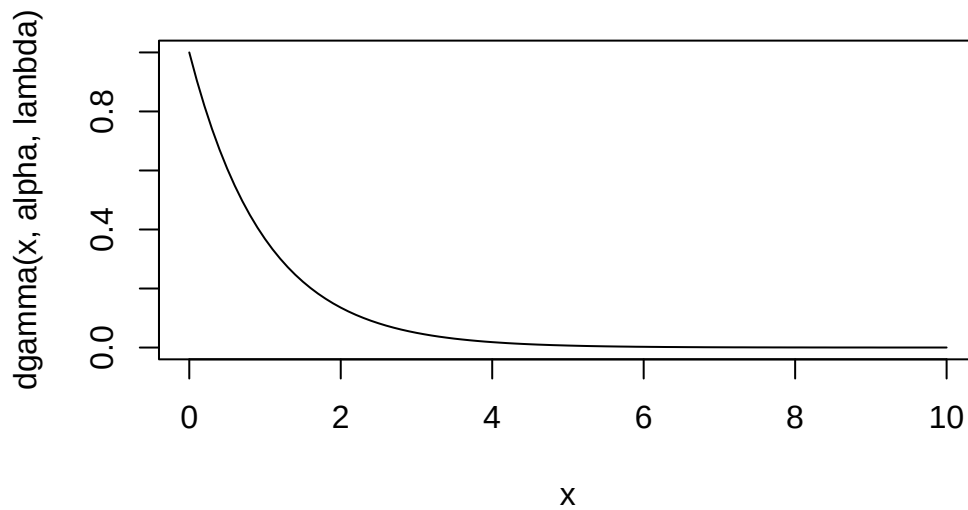
On peut aussi exécuté le chunk en ligne pour rechercher le resultat qui est ici : 2.997

### Application Shiny - La Loi Gamma

**\*\* Rappel \*\* :**

La loi de Gamma est une distribution de probabilité continue qui modélise des variables aléatoires réelles positives. Elle est utilisée pour décrire divers phénomènes, tels que la durée de vie d'un système ou le temps entre des événements.

```
alpha = 1  
lambda = 1  
curve(dgamma(x,alpha,lambda), 0, 10)
```



### Lancement de l'application shiny

@<https://lapoulefolle64.shinyapps.io/Projet-Killani/>

### Analyse de Données : data = "mtcars"

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
library(kableExtra)
```

Attaching package: 'kableExtra'

The following object is masked from 'package:dplyr':

`group_rows`

```
# Réalisation d'une analyse exploratoire - stat desc
stat_desc <- mtcars |>
  select(mpg, cyl) |>
  summarise(
    across(everything(), list(mean = mean, sd = sd))
  ) |>
  round(2)

# présentation sous forme de tableau????
kable(stat_desc, caption = "Moyenne et Écart-type (MPG et Cylindres)") |>
  kable_styling(bootstrap_options = "striped")
```

TABLE 1 : Moyenne et Écart-type (MPG et Cylindres)

| mpg_mean | mpg_sd | cyl_mean | cyl_sd |
|----------|--------|----------|--------|
| 20.09    | 6.03   | 6.19     | 1.79   |

## Regréssion Linéaire

Nous cherchons à expliquer la consommation (mpg) par la puissance (hp), en utilisant le jeu de donnée mtcars

```
### Installation des packages
#install.packages("dplyr")
#install.packages("kableExtra")
#install.packages("stargazer")
#install.packages("ggplot2")
#install.packages("ggthemes")
#install.packages("tidyr")
```

```
library(stargazer)
```

Please cite as:

Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary Statistics Tables.

R package version 5.2.3. <https://CRAN.R-project.org/package=stargazer>

```
library(ggplot2)
library(ggthemes)

modele <- lm(mpg ~ hp, data = mtcars)

# Tableau de régression avec stargazer
stargazer(modele, type = "html", title = "Impact de la puissance sur la consommation")
```

Impact de la puissance sur la consommation

Dependent variable :

mpg

hp

-0.068\*\*\*

(0.010)

Constant

30.099\*\*\*

(1.634)

Observations

32

R2

0.602

Adjusted R2

0.589

Residual Std. Error

3.863 (df = 30)

F Statistic

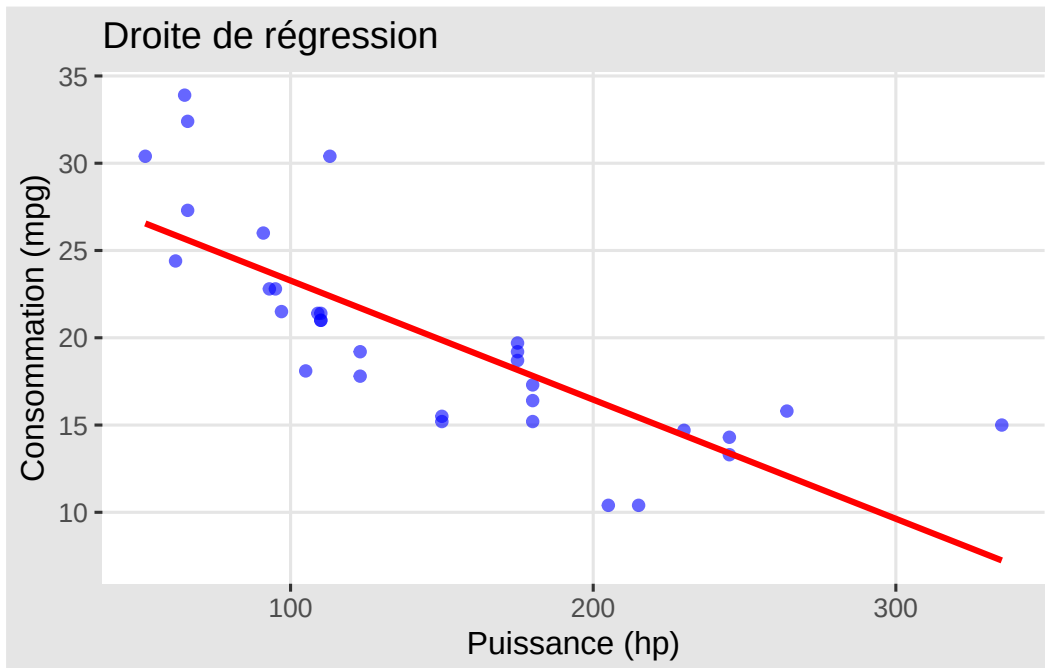
45.460\*\*\* (df = 1 ; 30)

Note :

$p < 0.1$  ;  $p < 0.05$  ;  $p < 0.01$

```
# Visualisation
ggplot(mtcars, aes(x = hp, y = mpg)) +
  geom_point(col = "blue", alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE, col = "red") +
  theme_igray() +
  labs(title = "Droite de régression", x = "Puissance (hp)", y = "Consommation (mpg)")
```

`geom\_smooth()` using formula = 'y ~ x'



## Analyse exploratoire

Tout d'abord on commence par afficher les noms des colonnes du dataset avec la fonction ***names()***, puis on crée un résumé statistique, de tel sorte à obtenir en sortie un tableau présentant des stats (moyenne et écart-type) des variables mpg et cyl

```
voiture <- mtcars
```

```
voiture|>names() ### pour voir le nom de chaque colonne
```

```
[1] "mpg"  "cyl"  "disp" "hp"   "drat" "wt"   "qsec" "vs"   "am"   "gear"
[11] "carb"
```

```
voiture |> select(mpg, cyl) |> summarise(mean_mpg = mean(mpg), mean_cyl = mean(cyl), sd_mpg = sd(mpg), sd_cyl = sd(cyl))
```

On transforme les résultats statistiques en une matrice structurée, en entrant manuellement les valeurs (moyenne, écart-type)

```
stat_desc |> matrix(c(20.02, 6.19, 6.03, 1.79), nrow = 2, ncol = 2,
                    dimnames = list(c("mean", "sd"), c("mpg", "cyl"))) |> kable() -> matrice
matrice
```

|      | mpg   | cyl  |
|------|-------|------|
| mean | 20.09 | 6.19 |
| sd   | 6.03  | 1.79 |

## Regression en Latex

```
#install.packages("stargazer")
library(stargazer)

modele = lm(mpg ~ hp, data = voiture)

### Pour faire le tableau
stargazer(modele,
           type = "text",
           title = "Modèle de régression du mpg expliqué par le hp",
           dep.var.labels = "mpg",
           covariate.labels = "hp",
           digits = 3)
```

## Modèle de régression du mpg expliqué par le hp

Dependent variable:

-----  
mpg

|          |                   |
|----------|-------------------|
| hp       | -0.068*** (0.010) |
| Constant | 30.099*** (1.634) |

Observations 32

R2 0.602

Adjusted R2 0.589

Residual Std. Error 3.863 (df = 30)

F Statistic 45.460\*\*\* (df = 1 ; 30)

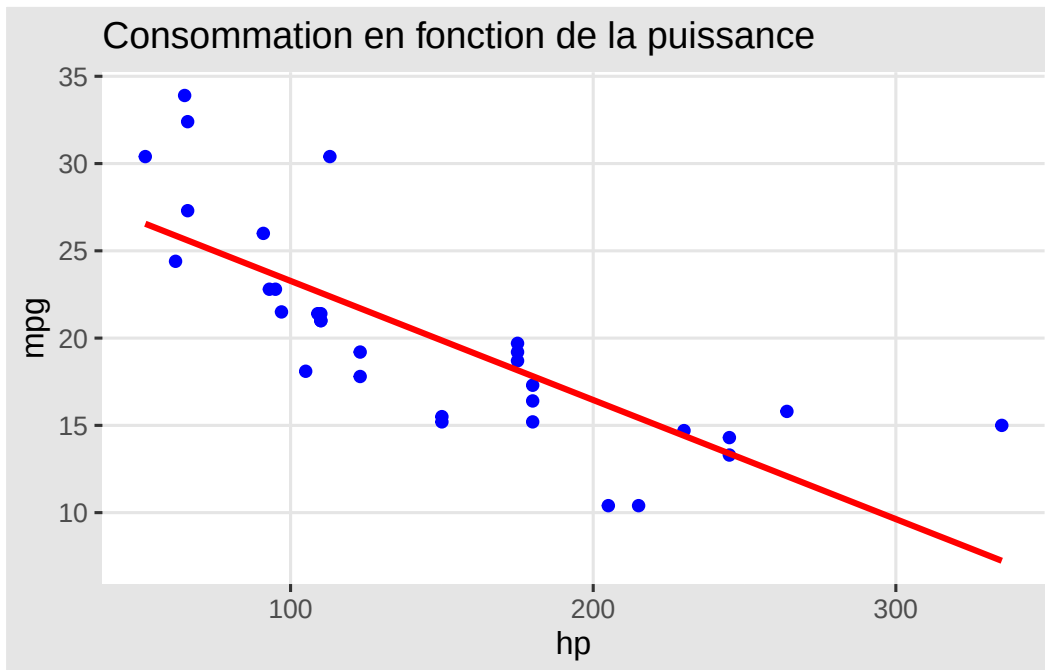
===== Note :  
 $p < 0.1$  ;  $p < 0.05$  ;  $p < 0.01$



## Droite de Regression

```
voiture |> ggplot(aes(hp, mpg)) + theme_igray() + ggtitle("Consommation en fonction de la pu
```

```
`geom_smooth()` using formula = 'y ~ x'
```



## Travail avec les données réel

On importe les données historiques du jeu Keno, puis on transforme les données du format **large** (c'est à dire une une colonne par boule) au format **long** pour faciliter notre analyse

```
#install.packages("tidyr")  
#install.packages("lubridate")  
library(tidyr)  
library(lubridate)
```

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

```
library(tidyr)
library(dplyr)

df = read.csv2("keno_202511 6 (1).csv")

## on supprime des variable
df = df[,-c(1, 3,20,21,22,23)]

# On convertit la date PUIS on bascule au format long
df1 <- df |>
  mutate(date_de_tirage = dmy(date_de_tirage)) |>
  pivot_longer(
    cols = !date_de_tirage,
    names_to = "boule",
    values_to = "valeur",
    names_prefix = "boule"
  )

df1
```

```
# A tibble: 1,600 x 3
  date_de_tirage boule valeur
  <date>         <chr>  <int>
1 2026-02-10     1      12
2 2026-02-10     2      13
3 2026-02-10     3      17
4 2026-02-10     4      20
5 2026-02-10     5      22
6 2026-02-10     6      23
7 2026-02-10     7      26
8 2026-02-10     8      27
9 2026-02-10     9      39
10 2026-02-10    10      42
# i 1,590 more rows
```

```
# Vérification de la date la plus récente
max(df1$date_de_tirage)
```

```
[1] "2026-02-10"
```

## Tableau des fréquences

```
library(scales)

# Création du tableau des fréquences et pourcentages
tableau_frequence <- df1 |> group_by(valeur) |> summarise(Frequence = n()) |>
  mutate(Pourcentage = percent(Frequence / sum(Frequence), accuracy = 0.01))

tableau_frequence
```

```
# A tibble: 56 x 3
  valeur Frequence Pourcentage
  <int>   <int>   <chr>
1     1     30 1.87%
2     2     26 1.62%
3     3     28 1.75%
4     4     32 2.00%
5     5     17 1.06%
6     6     21 1.31%
7     7     37 2.31%
8     8     23 1.44%
9     9     33 2.06%
10    10     30 1.87%
# i 46 more rows
```

## La probabilité qu'une boule sorte

### test de khi deux

Pour déterminer si les écarts de fréquences observés sont dus au hasard ou à un biais, nous réalisons un test de conformité à une loi uniforme, en fait on va chercher à vérifier l'équiprobabilité, c'est à dire qu'on se pose la question de savoir si toutes les boules ont la même probabilité d'être tirées

Sous l'hypothèse  $H_0$  : Les boules ont toutes la même probabilité de sortir  $H_1$  : Les boules n'ont pas du tout la même probabilité de sortir

NB : Notons que si la  $p\_value < \alpha$  : on rejette l'hypothèse nulle

```
chisq.test(tableau_frequencies$Frequence)
```

Chi-squared test for given probabilities

```
data: tableau_frequencies$Frequence  
X-squared = 45.14, df = 55, p-value = 0.826
```

Le test de khi deux sortant une pvalue de 0,82, on conserve l'hypothèse d'équiprobabilité, donc aucune boule n'est statistiquement favorisé

### Simulation de Monté carlo

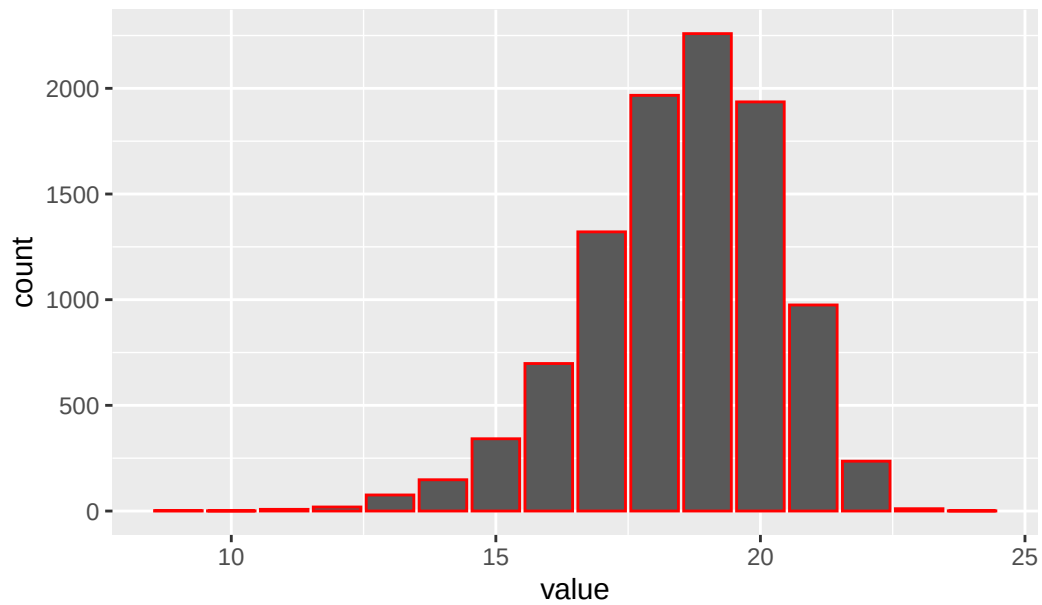
Afin de renforcer notre conclusion du khi deux, nous simulons 10 000 fois une série de 100 jours de tirages pour observer la distribution du minimum de fréquence (la boule qui sort le moins souvent).

```
set.seed(123)  
res <- c()  
for (i in 1:10000) {  
  # on simule en gros 100 jours de tirages (16 boules parmi 56)  
  ech <- replicate(100, sample(1:56, 16, replace = FALSE))  
  res <- c(res, min(table(ech)))  
}  
  
min(res)
```

```
[1] 9
```

```
library(ggplot2)  
  
as_tibble(res) |> ggplot(aes(value)) + geom_bar(col = "red") + ggtitle("Distribution empirique")
```

Distribution empirique des minimum de fréquence de sortie si



```
as_tibble(res) |> filter(value <= 17) |> summarise(Frequence = n())
```

```
# A tibble: 1 x 1
  Frequence
  <int>
1      2615
```

p value est forte donc on conserve l'hypothèse selon laquelle aucune boule n'est laissée